

PRACTICAL – 4

JavaScript DOM Manipulation, Event Handling, and UI Interactivity

AIM:

To add dynamic user-interface components to the StudentHub portal using JavaScript: a collapsible FAQ, a modal popup, a content slider, a notification banner, a hamburger menu and a light/dark theme switcher, with the theme choice remembered using localStorage.

Introduction:

In the earlier practicals, StudentHub was built as a set of static HTML pages styled with CSS. This practical makes the pages interactive. JavaScript selects elements from the Document Object Model (DOM), listens for user actions such as clicks and key presses, and then changes the page by updating text, attributes and visibility. All ten pages load one shared script, js/script.js, so the same behaviour is available across the portal.

Tools and Technology:

VS Code, JavaScript (ES6+), HTML5, CSS3, a web browser with developer tools (Console), Git and GitHub.

Features Implemented:

Feature	What it does	Where it appears
Light/dark theme switcher	Switches the colour theme and remembers the choice using localStorage.	Header of all ten pages
Hamburger menu	Shows or hides the navigation links on narrow (mobile) screens.	Header of all ten pages
Collapsible FAQ	Expands and collapses each answer when its question is clicked.	FAQ page
Modal popup	Opens a “Quick guide” dialog over the page and closes it again.	Home page
Notification banner	Shows a welcome notice that the user can dismiss.	Home page
Content slider	Moves through three “Campus highlights” slides using Previous and Next.	Home page

All styling for these components is kept in the existing css/style.css file, in the section marked “Practical 4”. The script and style rules only act on elements that exist on a given page, so no page produces errors for features that belong to another page.

Screenshots:

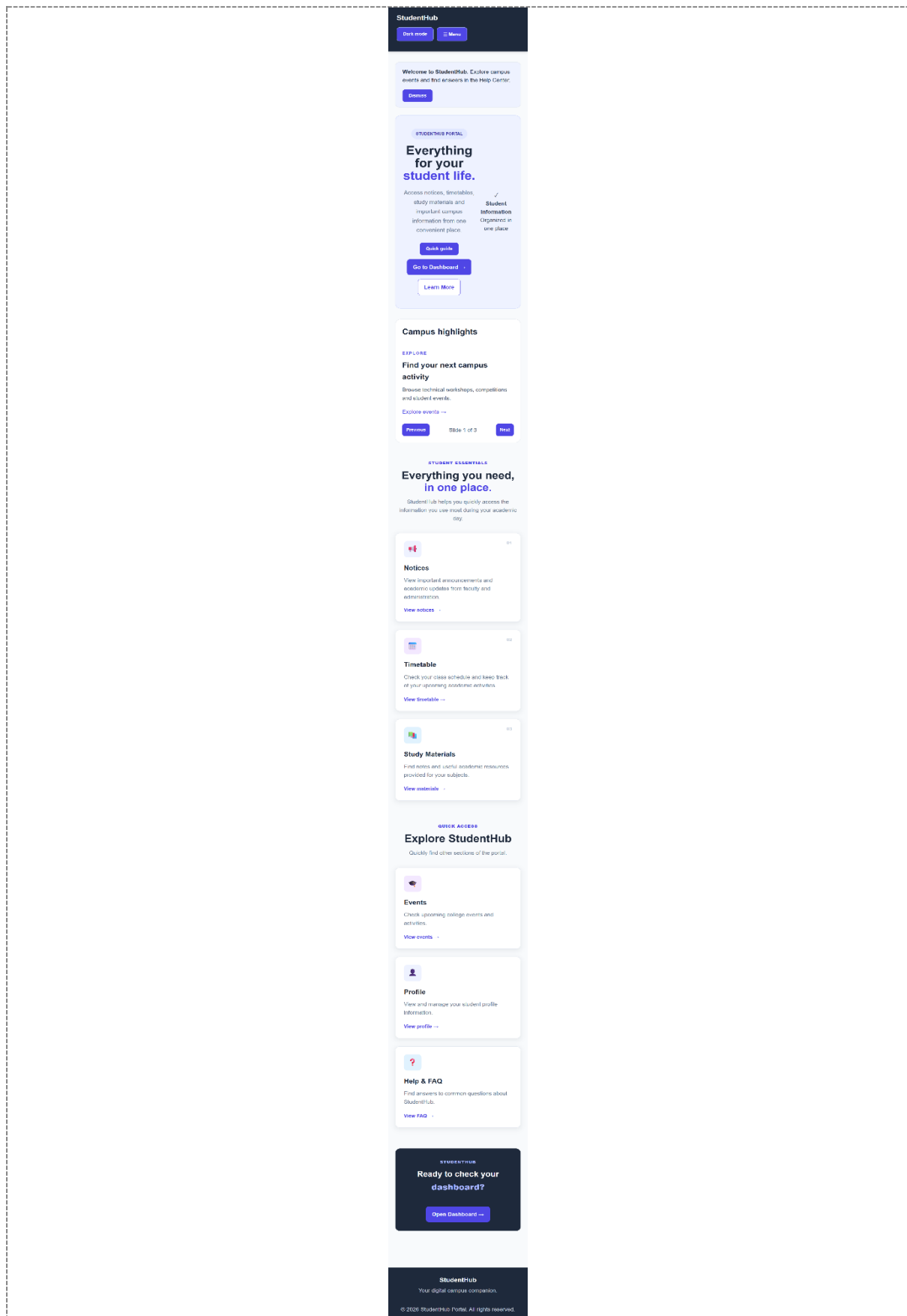


Figure 1(a): Home page in light mode

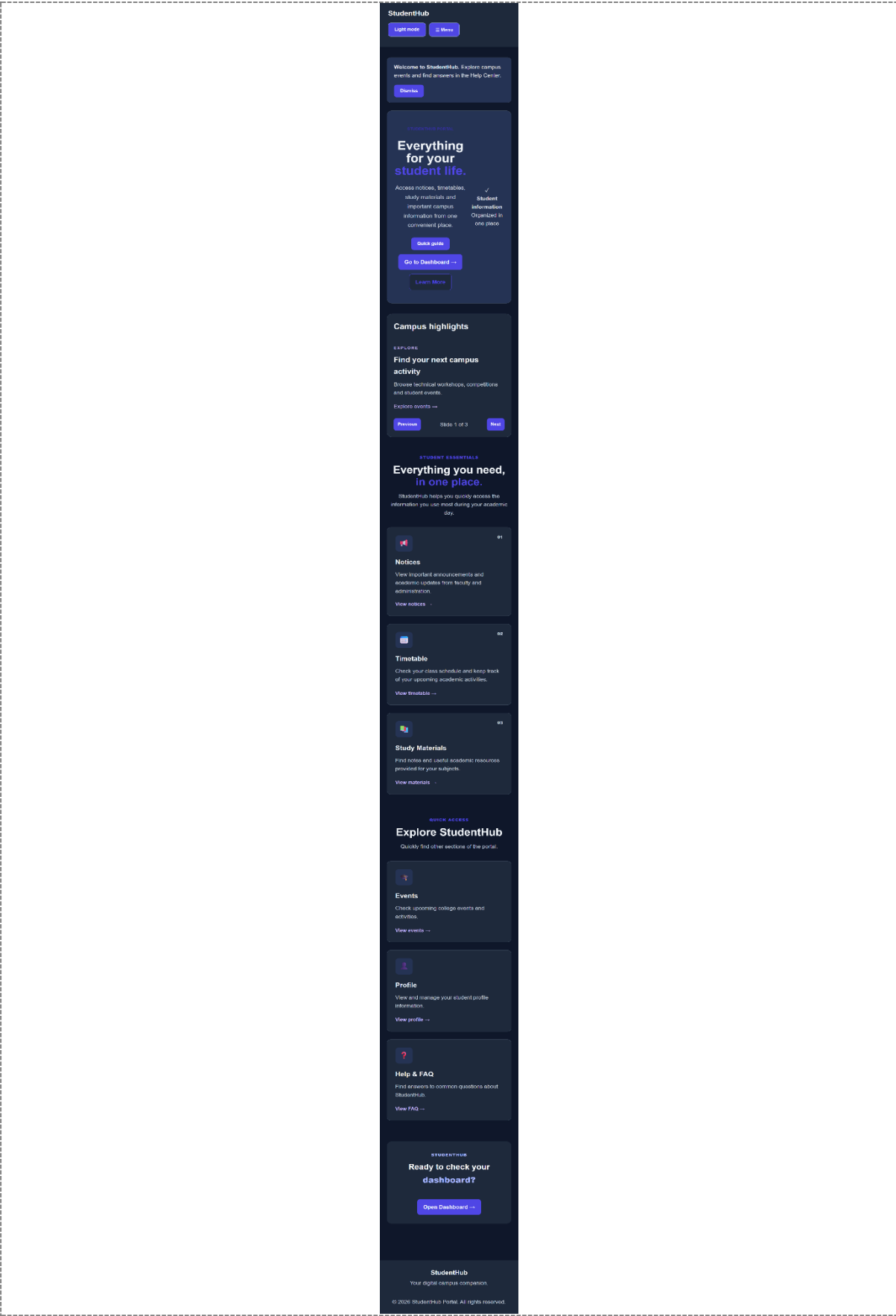


Figure 1(b): Home page in dark mode, still dark after refresh

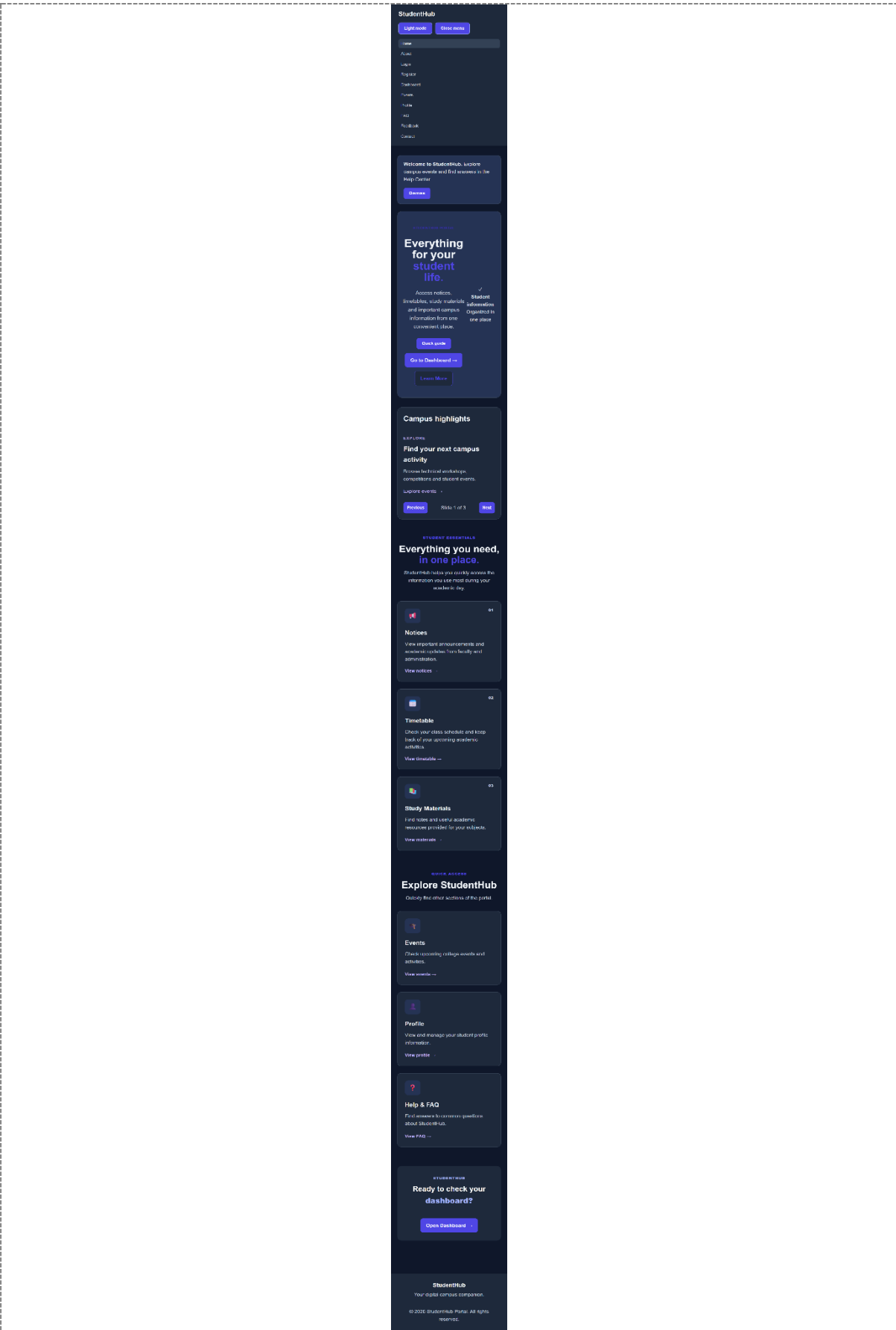


Figure 2(a): Mobile view with the menu closed

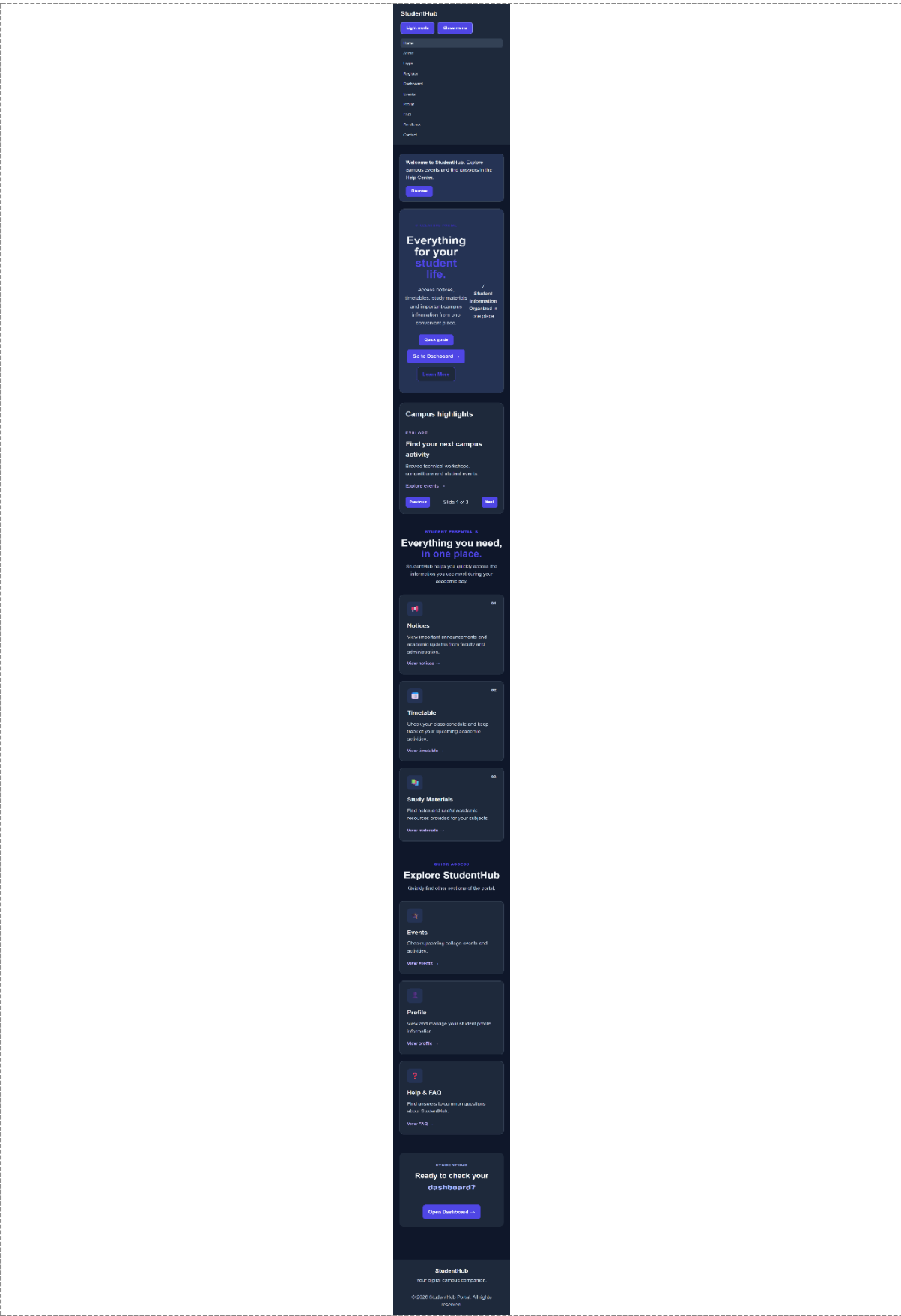


Figure 2(b): Mobile view with the menu open

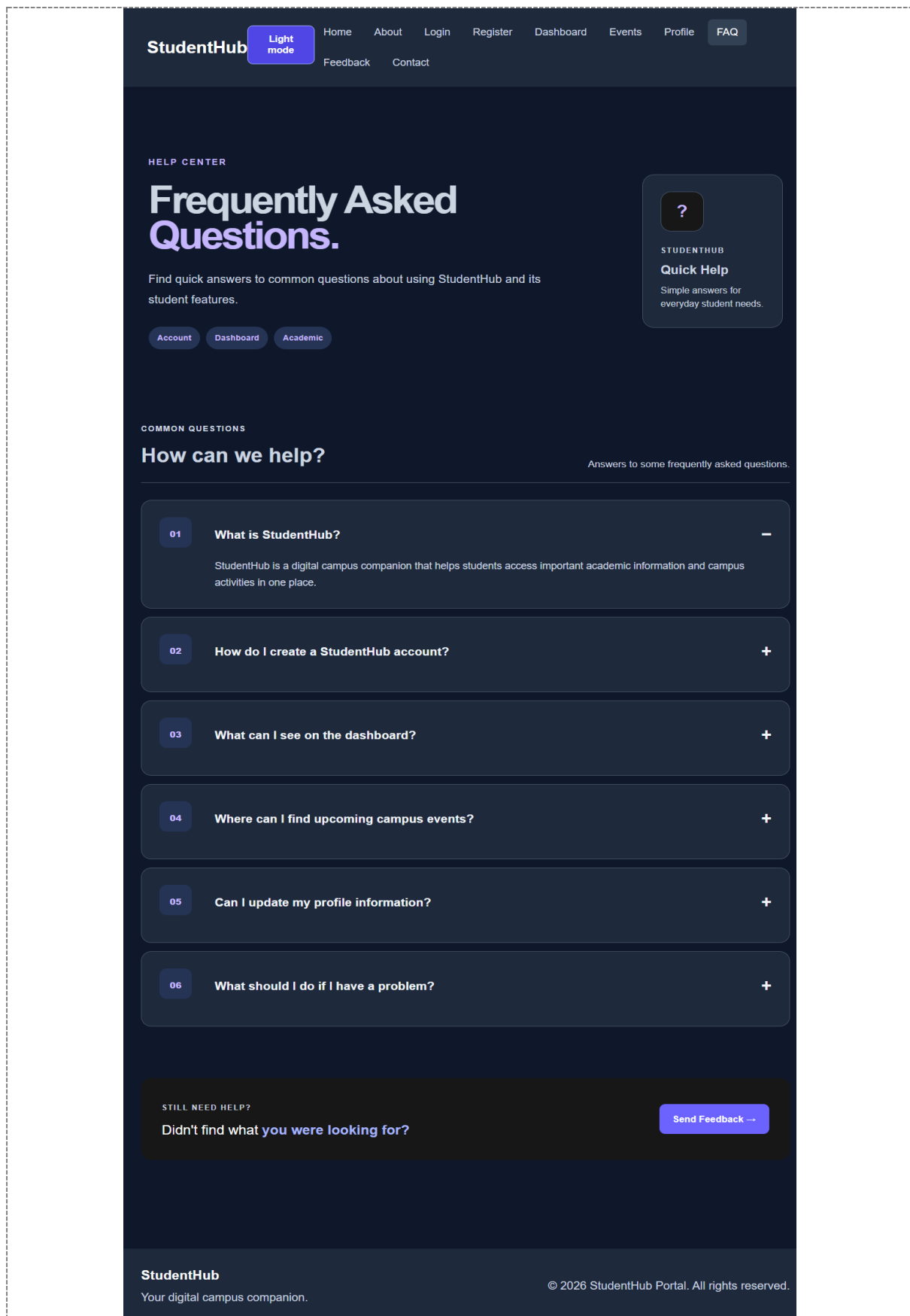


Figure 3: FAQ page showing an expanded answer

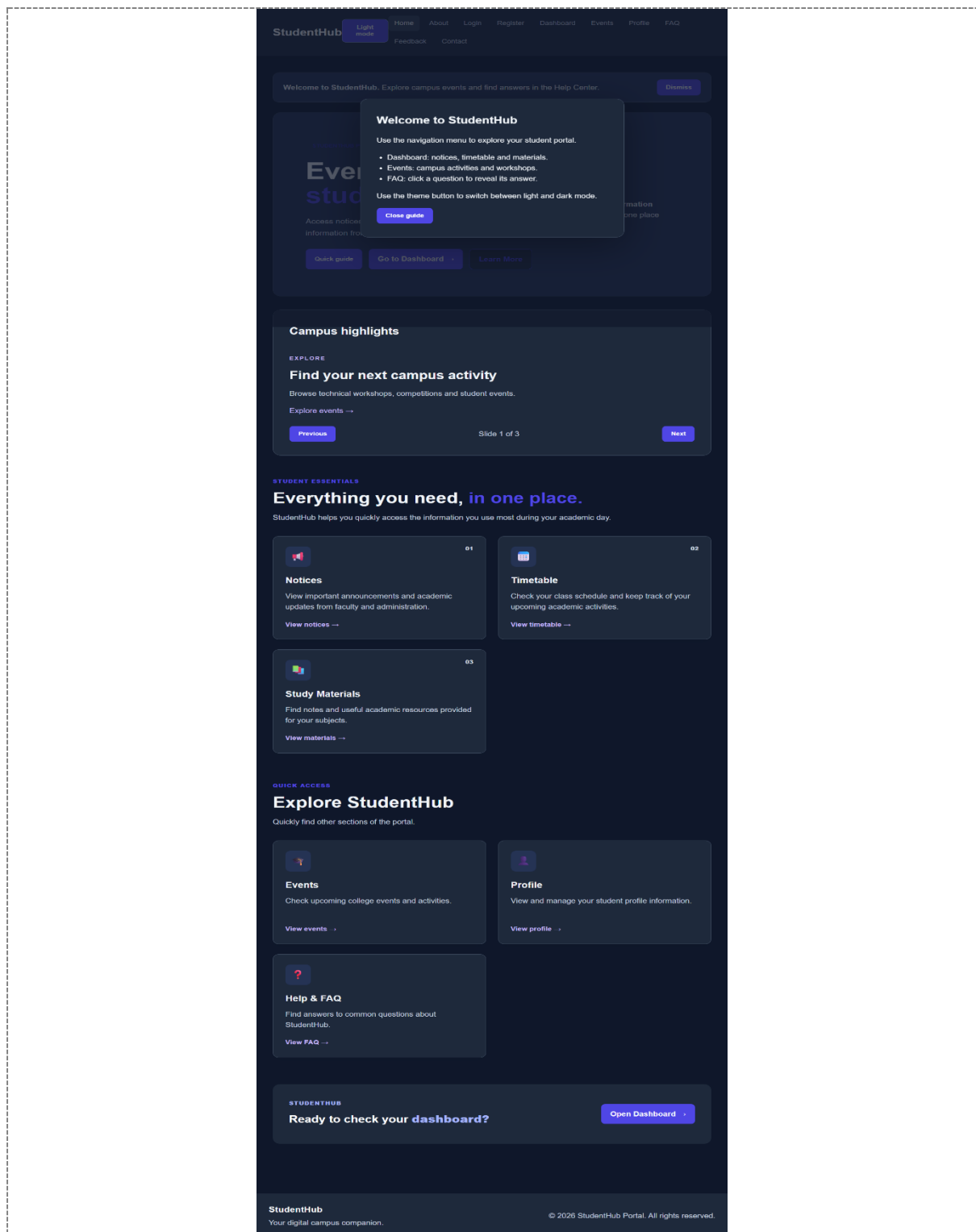


Figure 4: Modal popup displayed over the Home page

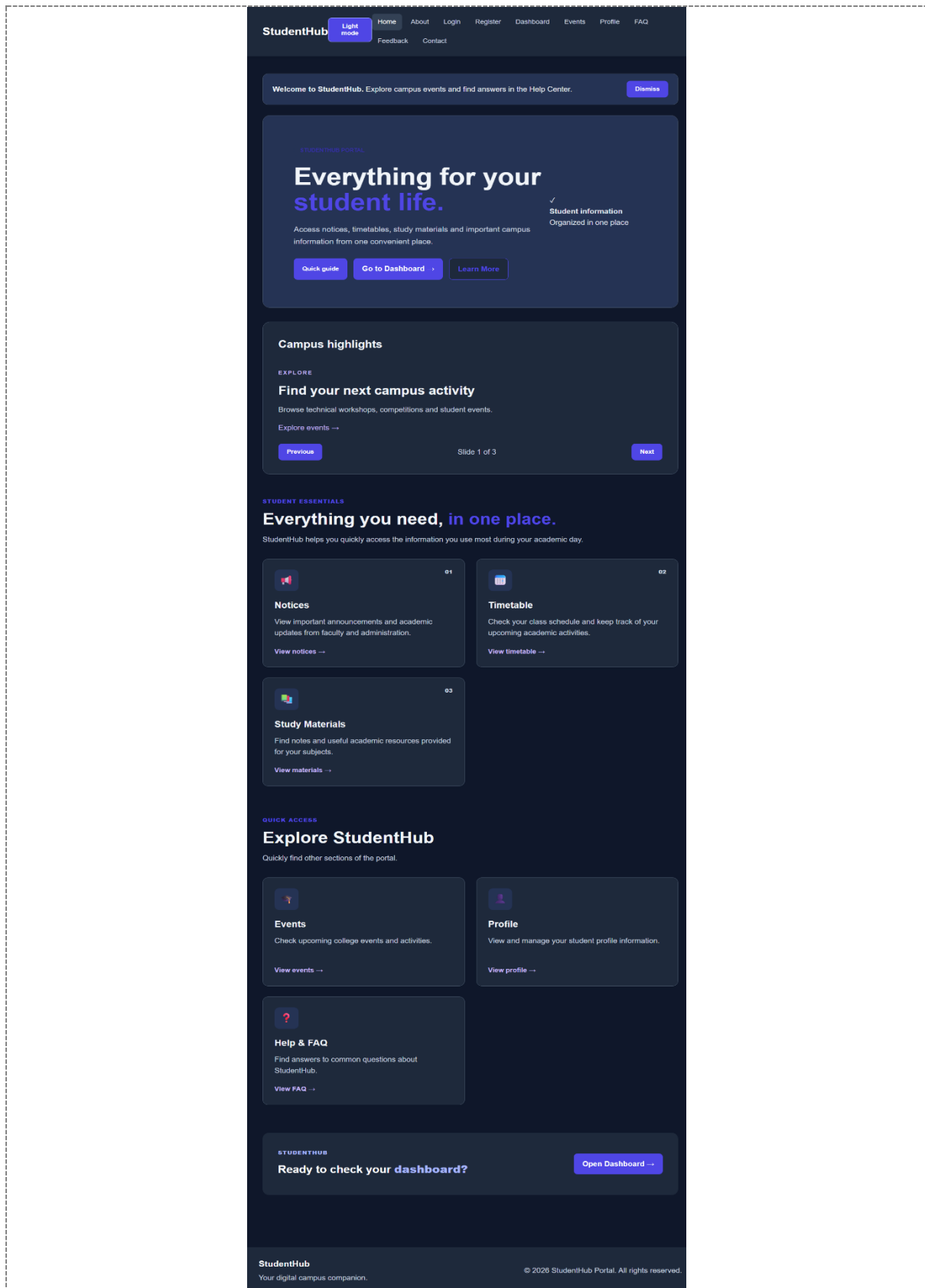


Figure 5(a): Notification banner visible

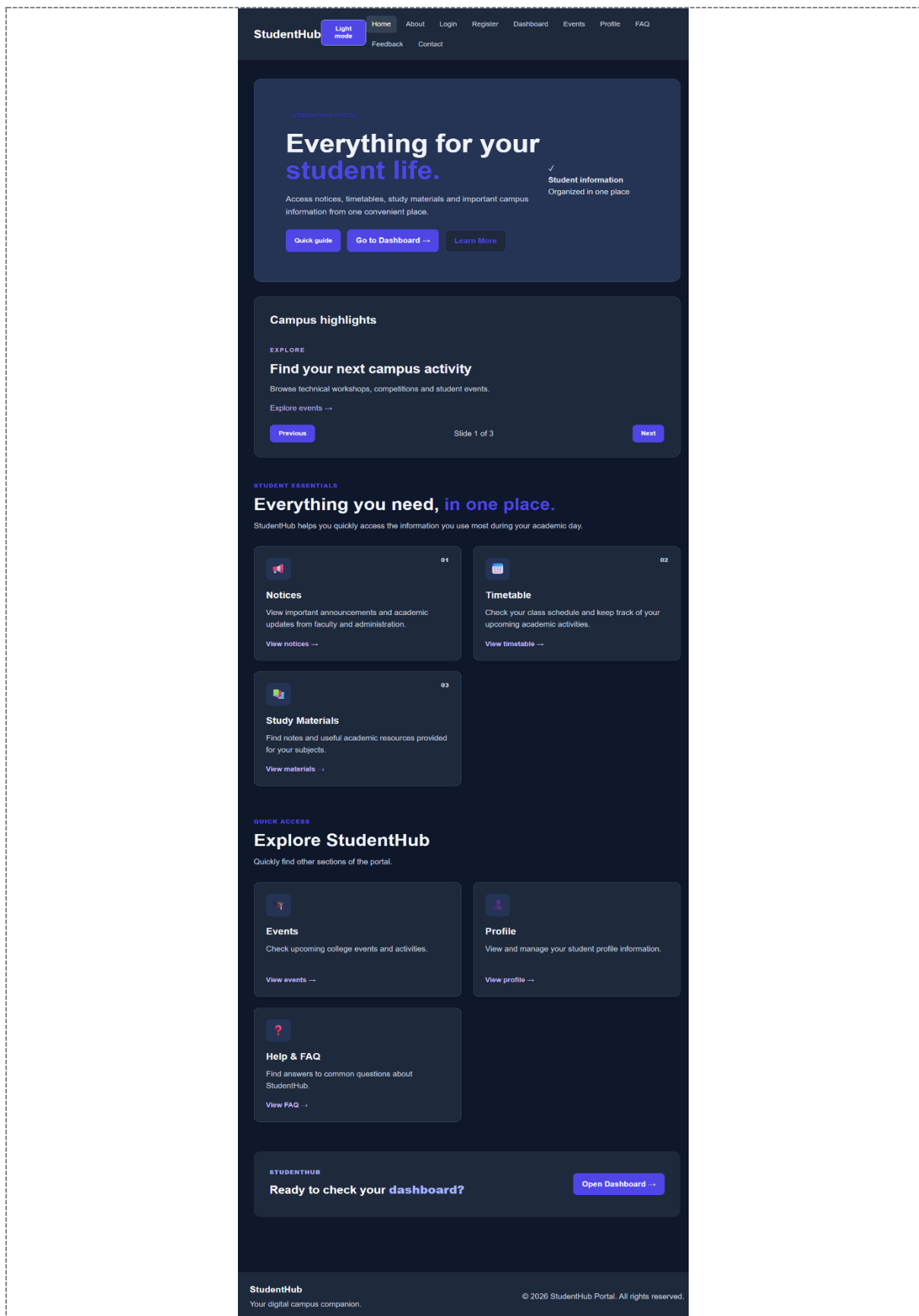


Figure 5(b): Notification banner dismissed

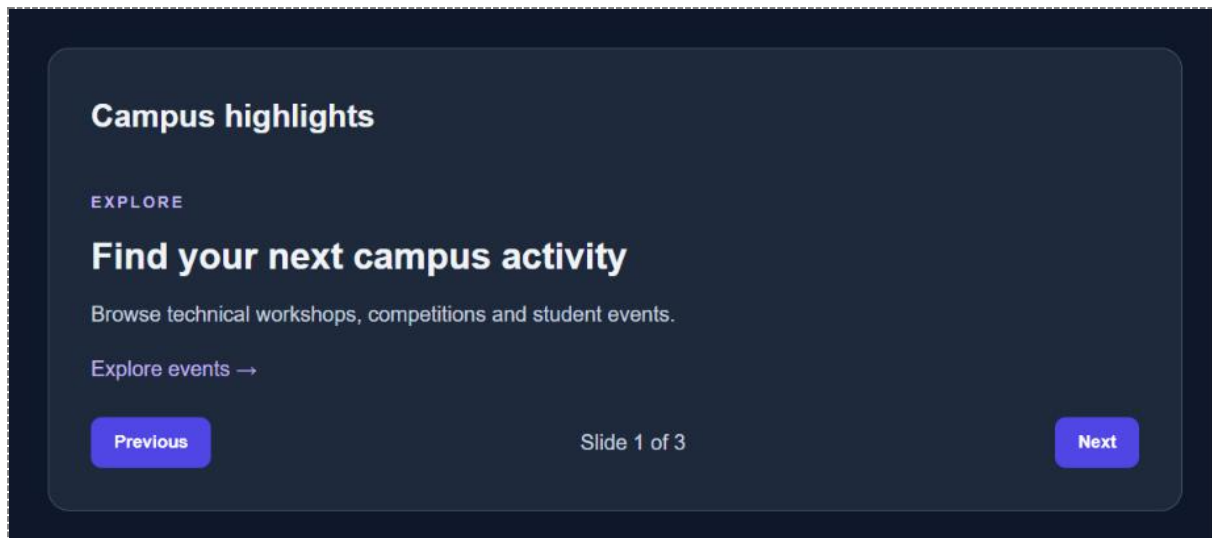


Figure 6(a): Slider showing the first slide

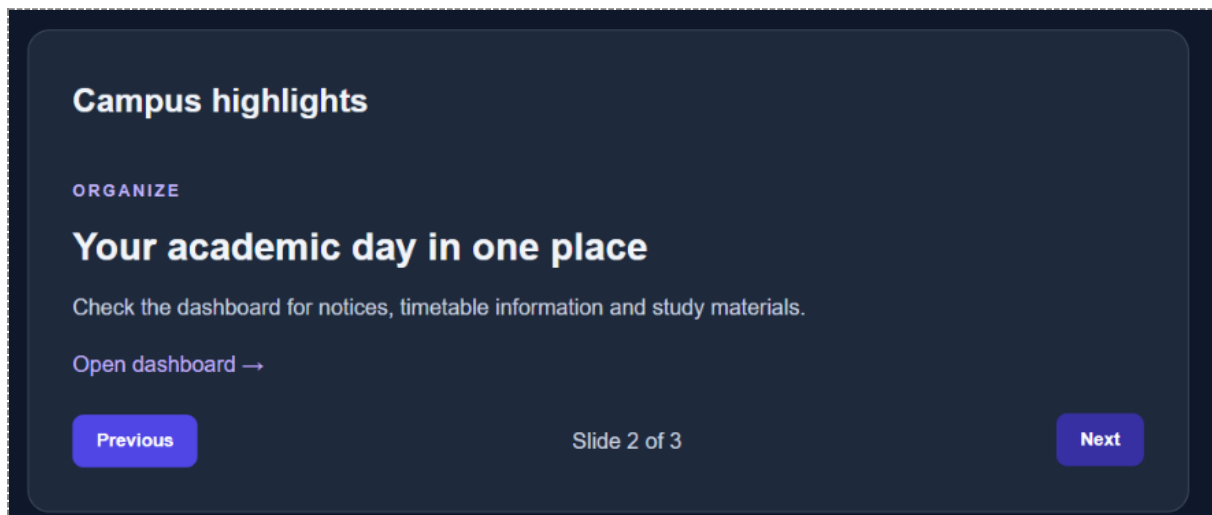


Figure 6(b): Slider after clicking Next

Key Questions and Answers

Q1. How are DOM elements selected and modified?

Elements are selected using `document.querySelector()` for a single element, `document.querySelectorAll()` for several, and `document.getElementById()` for a known id. Examples in the project are `querySelector('#theme-toggle')` and `querySelectorAll('.faq-toggle')`. Once selected, elements are modified by changing their `textContent`, the `hidden` property, attributes with `setAttribute()`, and data attributes with `dataset`. For example, the theme switcher changes `document.documentElement.dataset.theme`, and the FAQ changes an answer's `hidden` property.

Q2. Are event listeners attached correctly?

Yes. Listeners are attached with `addEventListener()` for click, keydown, change and close events. The script is loaded with the `defer` attribute, so the HTML is fully parsed before the script runs and every element already exists when a listener is attached. Each section first checks that its elements exist (for example if (guide && openGuide && closeGuide)), so the same script runs safely on all ten pages. Testing in the browser and Console showed that each control responds once per action.

Q3. Is localStorage used for remembering the theme choice?

Yes. When the user switches theme, `localStorage.setItem('studenthub-theme', theme)` saves it. On the next page load, `localStorage.getItem('studenthub-theme')` restores it before the theme is applied. Because `localStorage` belongs to the browser and stays after the tab is closed, the choice is kept across refreshes and across all StudentHub pages opened from the same address. The calls are placed inside `try/catch` so that the other components keep working if storage is blocked.

Q4. Does interactivity improve usability without breaking accessibility?

The features make the portal easier to use: dark mode reduces glare, the FAQ shortens long pages, the menu saves space on phones, and the slider shows more content in less space. Accessibility was preserved by:

- using native `<button>` and `<dialog>` elements, which support the keyboard and screen readers automatically;
- updating `aria-expanded`, `aria-pressed` and `aria-controls` whenever a control changes state;
- announcing slide changes with an `aria-live` status message;
- returning keyboard focus to a sensible control after the popup or notice closes;
- showing a clear focus outline on controls, and turning off animation for users who request reduced motion.

Post Laboratory Work:

Live demonstration of the StudentHub portal and explanation of the source code (`js/script.js`). The complete script (`js/script.js`) is available in the GitHub repository below.

GITHUB REPOSITORY LINK:

<https://github.com/DarshilRupani/studenthub-portal>

(FOR SOURCE CODE)

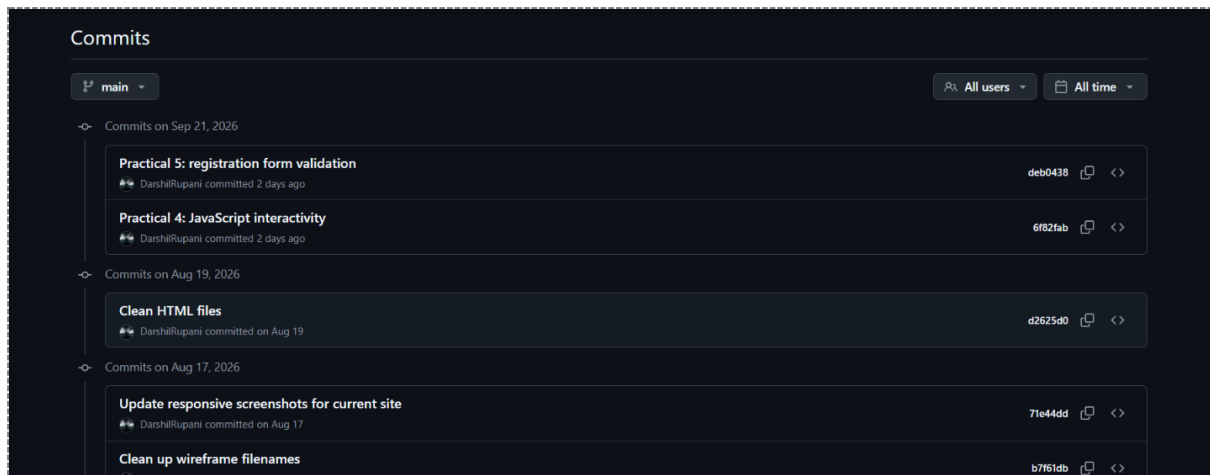


Figure 8: Practical 4 commit on GitHub (add after you push)

Conclusion:

JavaScript was used to add six interactive components to StudentHub: a theme switcher that remembers its setting, a hamburger menu, a collapsible FAQ, a modal popup, a dismissible notification and a content slider. DOM selection, event handling, class and attribute changes, localStorage and basic transitions were applied, and the components were built with native elements and ARIA attributes to stay accessible. The features were tested in the browser and the results are recorded above.